

FORM 2 - PROVISIONAL SPECIFICATION (VALUESKINS)

IP INDIA PROVISIONAL PATENT APPLICATION

TITLE OF THE INVENTION

Deterministic Creator-Brand Marketplace that can be attached to existing social media platforms, with Escrow-Gated Deal Workflows, Server-Authoritative Reputation Scoring, and Tamper-Evident Contract Architecture

FIELD OF THE INVENTION

This invention relates to a digital marketplace platform designed for negotiation, contracting, and performance verification between content creators and brands. Specifically, it addresses automated deal room state machines, legal contract generation tied to acceptance triggers, distributed reputation computation, and cryptographically verifiable deal evidence trails.

BACKGROUND OF THE INVENTION

PROBLEM STATEMENT

Current creator-brand engagement workflows suffer from structural failures:

1. No Verifiable Negotiation Record: Offers and counteroffers occur on Instagram DMs or email, creating dispute ambiguity. No cryptographic proof of agreement exists.
2. Middleman Extraction: Agencies capture 20-30% margins by controlling pricing discovery, contract templates, and performance guarantees.
3. Ghosting & Fraud: Creators deliver, brands withhold payment. Brands request free work. No binding contractual gates prevent cancellation or revision spam.
4. Pricing Opacity: No market-rate benchmarking per niche (profession, platform, content type, follower band). Creators underbid; brands overpay in some cases.
5. Reputation Portability: Trust signals locked to single platforms. No verifiable, exportable reputation score that works across ecosystems.
6. Escrow Uncertainty: No binding advance (upfront deposit) or milestone-based release logic. Payment contingent on brand goodwill.
7. Follower Fraud: Creators with bot-inflated followers win deals. No audit trail for fake engagement detection.

This invention eliminates all seven gaps via server-authoritative state machines, cryptographic contracts, deterministic reputation exports, and append-only deal records.

PRIOR ART GAP

While platforms like Stripe (payments), Upwork (bidding), and Instagram (discovery) exist:

- Stripe: Processes payments but enforces no contract logic or deal state.
- Upwork: Bidding-based, not opportunity-driven; no reputation export; brands control all state.
- Instagram: Offers discovery and messaging but zero legal bindingness; no escrow; no follower audit.

No existing system combines: (a) server-authoritative deal state, (b) auto-generated legally-binding contracts, (c) cryptographic evidence trails, (d) portable reputation passports, and (e) Meta-scale idempotency guarantees. This invention fills that gap.

SUMMARY OF THE INVENTION

The present invention provides a comprehensive system to replace agency middlemen by introducing:

1. **Deterministic Deal State Machine:** All deal state transitions (negotiation -> offer accepted -> contract signed -> in progress -> deliverables submitted -> completed) are server-enforced. Client cannot mutate state; server cryptographically verifies each transition.
1. **Auto-Generated Contracts Bound to Offer Acceptance:** The moment a creator or brand accepts an offer round, the system automatically generates a legally-binding contract (not a handshake). Both parties must e-sign the contract before deal funding begins. Contract content includes: exact amount, deliverable list, revision cap, exclusivity window, kill-fee percentage on brand cancellation, usage rights scope, and deadline.
1. **Tamper-Evident Contract Storage:** Each generated contract is assigned a SHA-256 content hash. Signatures store: (signer_id, timestamp, content_hash). If contract is later modified, signature becomes invalid - making tampering detectable.
1. **Append-Only Deal Room Message Log:** All negotiation occurs in deal room (not DMs). Every message includes server-assigned timestamp and sender ID derived from JWT (client cannot spoof). Messages are immutable at the database level (no UPDATE/DELETE permitted on the messages table).
1. **Portable Reputation Passport:** A creator's reputation (deal count, avg deal value, trust scores, testimonials, completion rate) is cryptographically signed with platform private key (Ed25519). The signature enables any brand to verify authenticity without querying the platform. Passports are versioned; old signatures remain valid.
1. **Deterministic Reputation Scoring:** Creator reputation is computed from: completed deal count, deal value band history, ghost-count penalty, contract_revision abuse count, and follower-audit score. Same inputs always produce the same score (no randomness, no time-decay).

1. Follower Audit Trail: Creator follower counts are audited by analyzing: engagement rate vs follower count anomaly (botted accounts show 0.1% engagement on 1M followers), account age distribution, and follower account health signals. Fake follower percentage is stored and surfaced in brand UX.
 1. Level-Based Matching Gate: Creators earn levels (1-5) deterministically from completed deal count + trust score. Opportunities specify required_level. Creators below required_level cannot apply (not in matching suggestions, not visible in opportunity detail).
 1. Advance Preference Compatibility: Before a deal room even opens, the system checks: if creator requires_advance=true AND brand offers_advance=false, the pair is not matched. This prevents dead-end negotiations.
 1. Rate Limits & Idempotency Keys: All mutation endpoints include idempotency keys. If a creator submits an application twice with the same key, the second request is silently ignored. Rate limits prevent apply-spam: max 10 applications per creator per 24h.
 1. Version-Stamped Market Rate Benchmarks: Pricing benchmarks (p25, median, p75 per categoryxplatformxcontent_typexlevel) are recomputed monthly from completed deals and stored with version number. Old versions queryable forever. Brands and creators use versioned benchmarks for negotiation.
 1. Credit Line System: Creators with completed deals + trust score >= threshold can draw advances against a credit line. Repayment auto-deducts from deal completion payout. Deterministic credit_score = f(deal_count, avg_deal_value, trust_score, months_active).
-

DETAILED DESCRIPTION OF THE INVENTION

1. DATABASE SCHEMA (CORE ENTITIES)

USERS

- id BIGSERIAL PRIMARY KEY
- username TEXT UNIQUE NOT NULL
- auth_provider TEXT NOT NULL (e.g., 'instagram_oauth')
- auth_provider_id TEXT NOT NULL (Instagram user ID)
- avatar_uri TEXT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- last_login_at TIMESTAMP

PERSONAS

- id BIGSERIAL PRIMARY KEY
- owner_user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE
- role TEXT NOT NULL (enum: 'creator', 'brand')
- display_name TEXT NOT NULL

- avatar_uri TEXT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- last_active_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (owner_user_id, role) for quick role lookup

PROFESSIONS

- id BIGSERIAL PRIMARY KEY
- name TEXT UNIQUE NOT NULL (e.g., 'Software Engineer', 'Graphic Designer')
- category TEXT NOT NULL (e.g., 'Tech', 'Art', 'Finance')
- description TEXT
- Index: (category) for filtering by profession category

PERSONA_PROFESSIONS

- persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- profession_id BIGINT NOT NULL REFERENCES professions(id) ON DELETE RESTRICT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Primary Key: (persona_id, profession_id) to prevent duplicates

PERSONA_LEVELS

- persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- level INT NOT NULL DEFAULT 1 (range 1-5, deterministic from completed deals)
- real_score BIGINT NOT NULL DEFAULT 0 (reputation score, deterministically computed)
- completed_deal_count INT NOT NULL DEFAULT 0
- avg_deal_value_cents BIGINT NOT NULL DEFAULT 0
- ghosted_count INT NOT NULL DEFAULT 0
- last_computed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Primary Key: persona_id
- Rationale: Denormalized for fast lookups; recomputed on deal completion or via scheduled job.

OPPORTUNITIES

- id BIGSERIAL PRIMARY KEY
- brand_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- title TEXT NOT NULL
- description TEXT NOT NULL
- category TEXT NOT NULL (e.g., 'Development', 'Art', 'Marketing')
- reward_amount TEXT NOT NULL (stored as string for precision)
- reward_currency TEXT NOT NULL (e.g., 'USD', 'INR')
- required_level INT NOT NULL DEFAULT 1 (min creator level to apply)
- status TEXT NOT NULL DEFAULT 'open' (enum: 'open', 'in_progress', 'completed', 'cancelled')

- offers_advance BOOLEAN DEFAULT false (brand opted into offering advances)
- advance_pct SMALLINT (default advance % offered, e.g., 30)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (brand_persona_id, status) for creator browsing
- Index: (status, created_at DESC) for marketplace feed
- Index: (required_level) for level-gating

APPLICATIONS

- id BIGSERIAL PRIMARY KEY
- opportunity_id BIGINT NOT NULL REFERENCES opportunities(id) ON DELETE CASCADE
- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- status TEXT NOT NULL DEFAULT 'applied' (enum: 'applied', 'rejected', 'deal_room_opened')
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Primary Key: (opportunity_id, creator_persona_id) prevents duplicate applications
- Rationale: Allows idempotent resubmission (second application with same key returns success but doesn't duplicate)

DEAL_ROOMS

- id BIGSERIAL PRIMARY KEY
- opportunity_id BIGINT NOT NULL REFERENCES opportunities(id) ON DELETE CASCADE
- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- brand_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- status TEXT NOT NULL DEFAULT 'negotiation' (enum: 'negotiation', 'contract_pending', 'funded', 'in_progress', 'completed', 'cancelled')
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (creator_persona_id, status) for creator's active deals
- Index: (brand_persona_id, status) for brand's active deals
- Unique: (opportunity_id, creator_persona_id) prevents duplicate deal rooms

OFFER_ROUNDS

- id BIGSERIAL PRIMARY KEY
- deal_room_id BIGINT NOT NULL REFERENCES deal_rooms(id) ON DELETE CASCADE
- initiator_persona_id BIGINT NOT NULL REFERENCES personas(id) (who made the offer)
- amount_cents BIGINT NOT NULL (offer amount in cents)

- currency TEXT NOT NULL DEFAULT 'USD'
- deliverable_description TEXT NOT NULL
- advance_offered_pct SMALLINT (advance % in this round, e.g., 30)
- revision_cap INT DEFAULT 2 (max revisions creator can request)
- exclusivity_days INT DEFAULT 0 (how long brand can claim exclusivity post-completion)
- kill_fee_pct INT DEFAULT 0 (% of amount brand pays if they cancel)
- status TEXT NOT NULL DEFAULT 'pending' (enum: 'pending', 'accepted', 'rejected', 'countered')
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- expires_at TIMESTAMP (offer expires if not accepted within 7 days)
- Index: (deal_room_id, status) for deal room history
- Index: (status, expires_at) for cleanup job to auto-expire old offers

OFFER_ROUND_RESPONSES

- id BIGSERIAL PRIMARY KEY
- offer_round_id BIGINT NOT NULL REFERENCES offer_rounds(id) ON DELETE CASCADE
- responder_persona_id BIGINT NOT NULL REFERENCES personas(id)
- response_type TEXT NOT NULL (enum: 'accept', 'reject', 'counter')
- counter_amount_cents BIGINT (if counter)
- counter_advance_pct SMALLINT (if counter)
- counter_revision_cap INT (if counter)
- notes TEXT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (offer_round_id, response_type) to find acceptances quickly
- Rationale: Append-only; every response is a new row. Immutable negotiation history.

DEAL_ROOM_MESSAGES

- id BIGSERIAL PRIMARY KEY
- deal_room_id BIGINT NOT NULL REFERENCES deal_rooms(id) ON DELETE CASCADE
- sender_persona_id BIGINT NOT NULL REFERENCES personas(id) (derived from JWT, cannot be spoofed)
- message_text TEXT NOT NULL
- server_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL (NOT client-provided)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Constraint: NO UPDATE/DELETE allowed on this table (enforced in application layer)
- Index: (deal_room_id, created_at ASC) for paginated message fetch
- Rationale: Immutable log. This is the authoritative evidence for disputes.

CONTRACTS

- id BIGSERIAL PRIMARY KEY
- deal_room_id BIGINT NOT NULL REFERENCES deal_rooms(id) ON DELETE CASCADE
- offer_round_id BIGINT NOT NULL REFERENCES offer_rounds(id) (which offer round triggered this contract)
- amount_cents BIGINT NOT NULL
- currency TEXT NOT NULL
- deliverable_list TEXT NOT NULL (JSON array of deliverables)
- revision_cap INT NOT NULL
- exclusivity_days INT NOT NULL
- kill_fee_pct INT NOT NULL
- usage_rights_scope TEXT NOT NULL (e.g., 'exclusive_post', 'reusable_ugc', 'ambassador_content')
- deadline TIMESTAMP NOT NULL
- content_hash TEXT NOT NULL (SHA-256 of serialized contract content; used to detect tampering)
- pdf_url TEXT (signed URL to downloadable PDF)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- generated_at TIMESTAMP NOT NULL (timestamp when contract was auto-generated from offer acceptance)
- Index: (deal_room_id) to find contract by deal room
- Rationale: Auto-generated the moment offer_round is accepted. Both parties must sign before state progresses.

CONTRACT_SIGNATURES

- id BIGSERIAL PRIMARY KEY
- contract_id BIGINT NOT NULL REFERENCES contracts(id) ON DELETE CASCADE
- signer_persona_id BIGINT NOT NULL REFERENCES personas(id)
- content_hash_at_sign TEXT NOT NULL (the SHA-256 of contract at sign time; must match contract.content_hash)
- signature_timestamp TIMESTAMP NOT NULL (server-assigned, not client)
- signed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Primary Key: (contract_id, signer_persona_id) prevents duplicate signatures
- Rationale: If both parties have signed, deal can progress. Mismatch in content_hash proves tampering.

FOLLOWER_AUDIT_RESULTS

- id BIGSERIAL PRIMARY KEY
- persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- platform TEXT NOT NULL (e.g., 'instagram', 'tiktok')

- follower_count INT NOT NULL (at time of audit)
- fake_follower_pct NUMERIC(5,2) (e.g., 12.5 means 12.5%)
- engagement_authenticity_score INT NOT NULL (0-100, deterministic)
- signal_breakdown JSONB (e.g., {"bot_account_ratio": 0.15, "new_account_ratio": 0.08, "dormant_ratio": 0.03})
- audited_at TIMESTAMP NOT NULL
- valid_until TIMESTAMP NOT NULL (30 days from audited_at)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (persona_id, platform) to find latest audit
- Unique: (persona_id, platform, audited_at) prevents duplicate audits on same day

PRICING_BENCHMARKS

- id BIGSERIAL PRIMARY KEY
- category TEXT NOT NULL (profession category, e.g., 'Tech')
- platform TEXT NOT NULL (e.g., 'instagram')
- content_type TEXT NOT NULL (e.g., 'reel', 'story', 'post')
- level INT NOT NULL (creator level 1-5)
- p25_cents BIGINT NOT NULL (25th percentile)
- median_cents BIGINT NOT NULL (50th percentile)
- p75_cents BIGINT NOT NULL (75th percentile)
- version INT NOT NULL (timestamp-based version, e.g., 202403 for March 2024)
- computed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (category, platform, content_type, level, version) for lookups
- Primary Key: (category, platform, content_type, level, version) to version benchmarks

CREATOR_CREDIT_LINES

- id BIGSERIAL PRIMARY KEY
- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- limit_cents BIGINT NOT NULL (e.g., \$10,000 credit line)
- used_cents BIGINT NOT NULL DEFAULT 0
- credit_score INT NOT NULL DEFAULT 300 (0-1000, deterministic)
- status TEXT NOT NULL DEFAULT 'active' (enum: 'active', 'suspended', 'closed')
- updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Primary Key: creator_persona_id

CREDIT_ADVANCES

- id BIGSERIAL PRIMARY KEY
- deal_room_id BIGINT NOT NULL REFERENCES deal_rooms(id) ON DELETE CASCADE

- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- amount_cents BIGINT NOT NULL
- status TEXT NOT NULL DEFAULT 'pending' (enum: 'pending', 'issued', 'repaid', 'forfeited')
- repayment_auto_deduct BOOLEAN DEFAULT true (auto-repay from deal completion payout)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- issued_at TIMESTAMP
- repaid_at TIMESTAMP
- Index: (creator_persona_id, status) to find active advances

REPUTATION_EXPORTS

- id BIGSERIAL PRIMARY KEY
- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- version INT NOT NULL (snapshot version; old versions queryable forever)
- completed_deal_count INT NOT NULL
- avg_deal_value_cents BIGINT NOT NULL
- trust_scores JSONB NOT NULL (e.g., {"delivery": 95, "professionalism": 92, "communication": 88})
- testimonial_count INT NOT NULL
- completion_rate NUMERIC(5,2) (e.g., 98.5%)
- content_hash TEXT NOT NULL (SHA-256 of exported data)
- ed25519_signature TEXT NOT NULL (platform-signed with private key)
- exported_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (creator_persona_id, version DESC) to find latest export

DEAL_SUGGESTIONS

- id BIGSERIAL PRIMARY KEY
- brand_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- opportunity_id BIGINT NOT NULL REFERENCES opportunities(id) ON DELETE CASCADE
- match_score INT NOT NULL (0-100, deterministic from valu-skin match + other factors)
- match_factors JSONB NOT NULL (e.g., {"valueskin_match": 95, "level_fit": 80, "price_band_overlap": 85, "trust_score": 92, "advance_compatible": true})
- status TEXT NOT NULL DEFAULT 'pending' (enum: 'pending', 'viewed', 'deal_room_opened', 'rejected')

- advance_compatible BOOLEAN NOT NULL (true if creator's advance preferences match brand's advance settings)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (creator_persona_id, status) to surface "Brands Want You" feed
- Index: (brand_persona_id, status) to view suggested creators per opportunity

ADVANCE_PREFERENCES

- id BIGSERIAL PRIMARY KEY
- persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- persona_role TEXT NOT NULL CHECK (persona_role IN ('creator', 'brand'))
- brand_offers_advance BOOLEAN DEFAULT false (brand: "I will offer advances")
- brand_default_advance_pct SMALLINT (brand: default % to offer, e.g., 30)
- creator_requires_advance BOOLEAN DEFAULT false (creator: "I require an advance")
- creator_advance_pct SMALLINT (creator: desired % advance, e.g., 40)
- creator_advance_negotiable BOOLEAN DEFAULT true (creator: can negotiate down, or hard deal-breaker)
- updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Primary Key: persona_id

COMPLETED_DEALS

- id BIGSERIAL PRIMARY KEY
- deal_room_id BIGINT NOT NULL REFERENCES deal_rooms(id) ON DELETE CASCADE
- amount_cents BIGINT NOT NULL
- creator_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- brand_persona_id BIGINT NOT NULL REFERENCES personas(id) ON DELETE CASCADE
- completed_at TIMESTAMP NOT NULL
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- Index: (creator_persona_id) for user profile history
- Index: (completed_at DESC) for benchmark recomputation job
- Rationale: Used only for pricing benchmark and reputation scoring. Append-only.

2. STATE MACHINE: DEAL ROOM STATUS TRANSITIONS

```

negotiation -> (offer accepted) -> contract_pending
    v (both parties sign contract)
funded
    v (creator uploads deliverables + hash)
in_progress
    v (brand approves deliverables)
completed

# Cancellation paths:
negotiation -> cancelled (at any time, before acceptance)
contract_pending -> cancelled (brand cancels; triggers kill-fee calculation)
funded -> cancelled (same; kill-fee auto-triggered)
in_progress -> cancelled (same; full amount becomes kill-fee)

```

Each transition is server-enforced. No client can mutate this field directly.

3. CONTRACT AUTO-GENERATION & E-SIGNATURE

Flow:

1. Creator and brand exchange offer/counter-offer in deal room.
2. One party clicks "Accept" on an offer_round.
3. Server detects acceptance (response_type = 'accept') and immediately triggers POST /contracts/generate.
4. System generates contract from offer_round data (amount, deliverables, revision_cap, kill_fee, exclusivity, usage rights, deadline).
5. Contract is serialized to JSON, SHA-256 hashed, and stored.
6. Both parties receive link to review contract (e-signature popup in UI).
7. Creator signs: inserts row into contract_signatures with (contract_id, creator_persona_id, content_hash_at_sign, signature_timestamp).
8. Brand signs: inserts row into contract_signatures with (contract_id, brand_persona_id, content_hash_at_sign, signature_timestamp).
9. Once both have signed AND content_hash matches contract.content_hash, deal_room.status transitions to funded.
10. If contract is not signed within 7 days, offer expires and deal_room reverts to negotiation.

Tampering Prevention:

- If contract.content_hash contract_signatures.content_hash_at_sign, signature is invalid.
- UI shows warning: "Contract was modified after signature."
- Deal cannot proceed; parties must re-sign.

4. DETERMINISTIC REPUTATION SCORING

Level Assignment (1-5):

```

completed_deals <= 1 -> level 1 (Newcomer)
completed_deals 2-5 -> level 2 (Rising)
completed_deals 6-20 -> level 3 (Established)
completed_deals 21-50 AND trust_score >= 85 -> level 4 (Expert)
completed_deals > 50 AND trust_score >= 92 -> level 5 (Legend)

```

Trust Score Computation (0-100):

```

base_score = 50

+ (completed_deals * 0.5)                # reward consistent delivery
+ (ghosted_count == 0 ? 20 : -ghosted_count * 15) # penalize ghosting
+ (contract_revision_abuse < 3 ? 15 : -20) # penalize revision spam
+ (follower_audit_score * 0.2)           # penalize fake followers
+ (testimonial_avg_rating / 5 * 10)      # reward positive feedback
+ (consistency_bonus_for_on_time_delivery)

capped at 100

```

Computation Time: Triggered on: (a) deal completion, (b) monthly recomputation job at 2 AM UTC, (c) manual admin refresh.

Storage: Stored in persona_levels.real_score to avoid re-computation on every marketplace load.

5. PORTABLE REPUTATION PASSPORT

Export Endpoint: GET /reputation/export?persona_id={id}

Response:

```

{
  "version": 1,
  "creator_persona_id": 123,
  "completed_deal_count": 15,
  "avg_deal_value_cents": 250000,
  "trust_scores": {
    "delivery": 95,
    "professionalism": 92,
    "communication": 88,
    "overall": 92
  },
  "testimonial_count": 14,
  "completion_rate": 98.7,
  "content_hash": "sha256:abc123...",
  "ed25519_signature": "base64:xyz789...",
  "exported_at": "2024-03-07T14:30:00Z"
}

```

Verification:

- Brand calls GET /verify-reputation?content_hash=...&signature=...&persona_id=...
- Server recomputes content_hash from reputation data.
- Verifies signature using platform's Ed25519 public key.
- Returns: { "valid": true, "exported_at": "...", "data": { ...} }

Longevity: All versions stored in reputation_exports. Old signatures remain valid forever (no expiry).

6. FOLLOWER AUDIT & FAKE ENGAGEMENT DETECTION

Deterministic Signals:

1. Engagement Anomaly: If follower_count = 1M but avg_likes = 2k (0.2% engagement), bot-like.
2. Account Age Distribution: If 60%+ followers created in last 3 months, suspicious.
3. Dormancy Ratio: If X% of followers have 0 posts in 6+ months, likely bots.
4. Profile Completion: If X% of followers have no bio, no posts, likely bots.

Score Calculation:

```
signal_breakdown = {  
  "bot_account_ratio": X,          # % of followers detected as bot-like  
  "new_account_ratio": Y,          # % created in last 90 days  
  "dormant_ratio": Z,              # % with 0 posts in 6+ months  
  "profile_completion_ratio": W    # % with <50% profile filled  
}  
  
fake_follower_pct = (bot_account_ratio + new_account_ratio*0.5 + dormant_ratio*0.3)  
  
engagement_authenticity_score = 100 - (fake_follower_pct + dormancy_penalty + new_account_penalty)
```

Storage: Cached in follower_audit_results for 30 days. Re-audit on demand (rate-limited to 1 per persona per 30 days).

Exposure: Brands see engagement_authenticity_score on creator profile. Score < 60 triggers "High fake engagement" badge.

7. LEVEL-GATED MATCHING

Constraint: Opportunity specifies required_level INT (e.g., 3).

Matching Filter: Creator's level < required_level -> NOT matched.

- Creator cannot see opportunity in marketplace browse.
- Creator is not in deal_suggestions feed.
- If creator tries direct URL to opportunity detail, returns HTTP 403 Forbidden.

Incentive: Creators are motivated to complete deals to level up and unlock higher-paying opportunities.

8. ADVANCE PREFERENCE COMPATIBILITY

Model:

```
advance_preferences.creator_requires_advance = true
advance_preferences.creator_advance_negotiable = false
-> HARD CONSTRAINT: brand must offer advance or deal won't open

advance_preferences.brand_offers_advance = false
-> Brand is opted out of offering advances

deal_suggestions.advance_compatible = true
-> This creator-opportunity pair passed advance compatibility check
```

Matching Logic:

```
if creator.requires_advance AND NOT creator.advance_negotiable
  AND NOT brand.offers_advance:
  don't_match = true
  advance_compatible = false
else:
  advance_compatible = true
```

Benefit: Eliminates wasted time. Creators looking for advances don't see brands that don't offer them.

9. RATE LIMITING & IDEMPOTENCY

Application Rate Limit:

- Max 10 applications per creator per 24h rolling window.
- Checked at POST /applications handler.
- Returns HTTP 429 with retry-after header.

Idempotency Keys:

- All mutation endpoints accept optional Idempotency-Key header (UUID).
- Server stores (endpoint, idempotency_key, response) in cache (Redis, TTL 24h).
- If same key submitted twice, return cached response (silently ignore re-submission).
- Application endpoints use (creator_persona_id, opportunity_id) as implicit idempotency key.

10. MARKET RATE BENCHMARKS (VERSIONED)

Recompute Job:

```
-- Monthly (1st of month, 2 AM UTC)
INSERT INTO pricing_benchmarks
SELECT
  prof.category,
  'instagram',
  'reel',
  pl.level,
  percentile_cont(0.25) WITHIN GROUP (ORDER BY cd.amount_cents),
  percentile_cont(0.50) WITHIN GROUP (ORDER BY cd.amount_cents),
  percentile_cont(0.75) WITHIN GROUP (ORDER BY cd.amount_cents),
  202403 as version,
  NOW()
FROM completed_deals cd
JOIN deal_rooms dr ON dr.id = cd.deal_room_id
JOIN personas p ON p.id = dr.creator_persona_id
JOIN persona_professions pp ON pp.persona_id = p.id
JOIN professions prof ON prof.id = pp.profession_id
JOIN persona_levels pl ON pl.persona_id = p.id
WHERE cd.completed_at >= NOW() - INTERVAL '90 days'
GROUP BY prof.category, pl.level;
```

Usage:

- Creator wants to know "what am I worth?" -> Query latest version of benchmarks.
- Brand wants to see "what do experts in my niche typically charge?" -> Same query.
- If benchmark version changes, old version is still queryable for historical comparison.

11. CREDIT LINE & ADVANCE LOGIC

Credit Score Determinism:

```
base_score = 300

+ min(completed_deals * 5, 200)      # max +200 for deal history
+ min(trust_score / 0.5, 200)         # max +200 for trust
+ min(months_active, 120)             # max +120 for tenure
+ (ghosted_count == 0 ? 50 : 0)       # +50 if zero ghosts
+ (on_time_delivery_rate * 100)       # bonus for punctuality

capped at 1000
```

Advance Issuance:

1. Creator in active deal with offer_round.advance_offered_pct = 30%.
2. Creator clicks "Draw Advance" in deal room.
3. System checks: can we draw from credit_line?
 - used_cents + advance_amount <= limit_cents?
 - creator credit_score >= 400?
1. If yes: insert row into credit_advances with status='pending'.
2. Emit event to Meta payment rail: { "event": "advance_issued", "creator_id": "...", "amount_cents": 50000, "deal_room_id": "...", "repayment_auto_deduct": true }
3. Meta processes advance (deposits funds to creator).

4. On deal completion: Creator payout = (amount_cents - advance_cents). Deduction auto-applied.
5. Update credit_advances.status = 'repaid' and credit_lines.used_cents -= amount_cents.

Rationale: Deterministic scoring prevents gaming. Auto-repay eliminates chasing creators for repayment.

CLAIMS

CLAIM 1: SERVER-AUTHORITATIVE DEAL STATE MACHINE

A system comprising:

- A PostgreSQL database with a deal_rooms table having status column constrained to enum values ('negotiation', 'contract_pending', 'funded', 'in_progress', 'completed', 'cancelled').
- A backend handler that validates state transitions: only certain transitions are allowed (e.g., negotiation -> contract_pending only if offer_round.status='accepted').
- Client cannot directly mutate deal_room.status; all mutations routed through API endpoints.
- Each transition produces audit trail in append-only deal_room_messages table with server-assigned timestamp.

CLAIM 2: CONTRACT AUTO-GENERATION TRIGGERED BY OFFER ACCEPTANCE

A system wherein:

- Upon detection of an offer_round status change to 'accepted', the backend automatically calls a contract generation handler.
- The contract is synthesized from offer_round fields: amount_cents, deliverable_description, advance_offered_pct, revision_cap, exclusivity_days, kill_fee_pct, deadline.
- The generated contract is assigned a SHA-256 content hash.
- Both creator and brand must e-sign the contract (via signature insertion into contract_signatures table) before deal state transitions to 'funded'.
- E-signatures include signer_persona_id, signature_timestamp (server-assigned), and content_hash_at_sign.
- If contract content is modified post-signature, content_hash mismatch is detected and deal halts.

CLAIM 3: APPEND-ONLY DEAL ROOM MESSAGE LOG WITH SERVER TIMESTAMP AUTH

A system comprising:

- An immutable deal_room_messages table with NO UPDATE/DELETE constraints at database level.
- Every message insertion includes sender_persona_id (derived from JWT, not client-provided).
- Server assigns server_timestamp at insertion time; client timestamp is rejected.
- Message log serves as authoritative evidence trail for dispute resolution (replaces Instagram DMs, email threads).

- Full message history exportable as JSON for platform moderation or legal review.

CLAIM 4: DETERMINISTIC REPUTATION SCORING WITH PORTABLE CRYPTOGRAPHIC

A system comprising:

- Reputation score computed deterministically from: completed_deal_count, avg_deal_value, trust_score (itself deterministic), testimonial count, on-time delivery rate.
- No randomness, no time-decay, no ML.
- Score computation triggered on deal completion, monthly recomputation, or manual refresh.
- Export endpoint returns JSON with Ed25519 platform signature.
- Signature remains valid forever (no expiry). Old exports queryable indefinitely.
- External entities (brands, other platforms) can verify authenticity without querying originating platform.

CLAIM 5: FOLLOWER AUDIT TRAIL WITH DETERMINISTIC FAKE ENGAGEMENT DETECTION

A system comprising:

- Follower audit computes signal_breakdown via: engagement_rate vs follower_count anomaly, account_age distribution, dormancy_ratio, profile_completion_ratio.
- Fake_follower_pct = f(signal_breakdown), deterministically.
- Audit cached for 30 days; re-audit rate-limited to 1 per persona per 30 days.
- Audit results surfaced in brand UX: creators with high fake engagement receive "High fake engagement" badge.
- Audit score factored into matching algorithm; low-score creators deprioritized in suggestions.

CLAIM 6: LEVEL-GATED OPPORTUNITY ACCESS WITH DETERMINISTIC LEVEL ASSIGNMENT

A system comprising:

- Creator level assigned deterministically: based on completed_deal_count and trust_score (both inputs themselves deterministic and immutable).
- Opportunity specifies required_level (e.g., 3).
- Creators with level < required_level: (a) cannot see opportunity in marketplace, (b) cannot appear in deal_suggestions for this opportunity, (c) cannot open deal room (API returns 403 Forbidden).
- Level re-assignment triggered on deal completion (same transaction that marks deal completed).
- No level decay over time; levels are monotonically increasing.

CLAIM 7: ADVANCE PREFERENCE COMPATIBILITY MATCHING

A system comprising:

- Advance_preferences table stores: (creator_requires_advance, creator_advance_pct, creator_advance_negotiable) and (brand_offers_advance, brand_default_advance_pct).

- Matching algorithm checks: if creator.requires_advance=true AND NOT creator.advance_negotiable AND NOT brand.offers_advance, pair is not matched.
- Deal_suggestions.advance_compatible flag indicates whether this creator-opportunity pair is compatible.
- Incompatible pairs are not surfaced in suggestions or marketplace browse, preventing time-wasted negotiations.

CLAIM 8: RATE LIMITING & IDEMPOTENT APPLICATION SUBMISSION

A system comprising:

- Application endpoint enforces per-creator rate limit: max 10 applications per 24h (checked against applications table).
- Idempotency: (creator_persona_id, opportunity_id) tuple is unique in applications table.
- Re-submission with same (creator_id, opportunity_id) is silently accepted (idempotent); no duplicate application created.
- Rate limit returns HTTP 429 with Retry-After header indicating seconds until next application allowed.

CLAIM 9: VERSION-STAMPED MARKET RATE BENCHMARKS

A system comprising:

- pricing_benchmarks table stores p25/median/p75 per (category, platform, content_type, level, version).
- Benchmarks recomputed monthly from completed_deals (last 90 days).
- Version field is timestamp-based (e.g., 202403 for March 2024).
- Old versions never deleted; all versions queryable.
- Brand and creator UI displays both current benchmark and previous version for comparison.
- Benchmark recompute is idempotent: same inputs always produce same output.

CLAIM 10: DETERMINISTIC CREDIT LINE SCORING & AUTO-REPAYING ADVANCES

A system comprising:

- Credit score computed deterministically: f(completed_deals, avg_deal_value, trust_score, months_active, on_time_rate, ghosted_count).
- No randomness, no ML, no time-decay.
- Credit line limit is function of score: score 600-700 -> \$2k limit, 700-800 -> \$5k, 800+ -> \$10k.
- Advance issuance: creator draws X% of deal amount (pre-contracted). System emits event to Meta payment rail.
- On deal completion, advance auto-repays: creator payout = (total_amount - advance_amount), with automatic deduction.
- All advance transactions logged in credit_advances table for audit trail.

CLAIM 11: APPEND-ONLY AUDIT TRAIL FOR ALL MUTATIONS

A system comprising:

- All mutations (application, offer, contract, signature, advance) produce immutable audit log entries.
 - Each log entry includes: mutation_type, actor_persona_id, affected_record_id, old_state, new_state, timestamp, idempotency_key.
 - Mutations are idempotent: same idempotency key + same mutation always produces same result.
 - Audit log queryable by platform moderators for dispute resolution.
 - Audit log never modified retroactively (DELETE/UPDATE forbidden).
-

ADVANTAGES OF THE INVENTION

1. Eliminates Middleman Extraction: By automating deal negotiation, contract generation, and escrow logic, agencies can no longer extract 20-30% margins.
 1. Verifiable Negotiation Record: All offers, counteroffers, and messages logged with server timestamp. No "I said, you said" disputes. Cryptographic signatures prove agreement.
 1. Binding Contracts at Scale: Contract generation is automatic and deterministic. Every deal has a legally-binding contract. Revision caps prevent endless back-and-forth.
 1. Escrow Certainty: Advance preferences, kill fees, and milestone-based releases are pre-contracted. Brands cannot unilaterally cancel without penalty.
 1. Market-Rate Transparency: Benchmarks per niche prevent both under-bidding (creators) and overpaying (brands). Versioned benchmarks enable historical comparison and trend detection.
 1. Portable Reputation: Creators own their reputation. Signature verification enables cross-platform trust without data silos.
 1. Fraud Prevention: Follower audit detects bots. Level-gating prevents unvetted creators from accessing high-value opportunities. Credit lines prevent overstretching.
 1. Platform Scalability: Deterministic, stateless design scales horizontally. No single points of failure. Append-only data model enables parallel writes.
 1. Regulatory Compliance: Immutable audit trail satisfies legal discovery, tax reporting (completed deals), and fraud investigation.
 1. Integration Safety: Design is Meta-scale compatible. No hardcoded globals, no platform-specific assumptions. Can be integrated into existing Creator Studio or other platforms without breaking changes.
-

FAILURE CASES & MITIGATION

CASE 1: CONTRACT NOT SIGNED WITHIN 7 DAYS

Mitigation: Scheduled job checks for `deal_rooms.status = 'contract_pending'` AND `now() > updated_at + 7 days`. Auto-reverts to 'negotiation'. Notifies both parties: "Contract signing expired. Please review and re-sign."

CASE 2: DELIVERABLE SUBMITTED BUT DOES NOT MATCH CONTRACT

Mitigation: Deliverable upload requires SHA-256 hash. If hash does not match `contract.deliverable_list` SHA-256, upload rejected. Brand can challenge: "Does not meet contract spec." Dispute escalation to platform moderation.

CASE 3: BRAND CANCELS MID-DELIVERY

Mitigation: If `deal_room.status = 'in_progress'` and brand initiates cancellation, `deal_room.status` -> 'cancelled'. Kill-fee automatically triggered: `brand owes kill_fee_pct * amount_cents`. Emitted as payout event to Meta payment rail.

CASE 4: CREATOR GHOSTS AFTER FUNDING

Mitigation: If `deal_room.status = 'funded'` for 14+ days with no deliverable upload, automatic escalation: (1) notify creator, (2) if no response after 48h, mark deal as 'completed' (no deliverable) and increment `ghosted_count`. Creator's reputation score drops. Credit line may be suspended.

CASE 5: OFFER ROUND EXPIRES

Mitigation: If `offer_round.expires_at < now()` and `offer_round.status = 'pending'`, scheduled job updates to `status='expired'`. Parties notified. New offer can be created.

CASE 6: ADVANCE REQUEST WHILE CREDIT LINE EXHAUSTED

Mitigation: Credit line check happens at request time. If `used_cents + advance_amount > limit_cents`, HTTP 400 returned: "Insufficient credit. Requested \$X, available \$Y."

CASE 7: REPUTATION EXPORT SIGNATURE INVALID

Mitigation: Verification endpoint checks `content_hash` and Ed25519 signature. If mismatch, returns { "valid": false, "reason": "signature_mismatch" }. Brand is warned not to trust the export.

CASE 8: FOLLOWER AUDIT CORRUPTED

Mitigation: Audit signals stored in `signal_breakdown` JSONB. If any signal is missing or null, audit is marked invalid and re-run. Fallback: assume `fake_follower_pct = 50%` (conservative) until fresh audit completes.

INTEGRATION WITH META PLATFORM

ValueSkins is designed as a platform capability that Meta can integrate without schema rewrites:

1. Identity: Uses Instagram OAuth (existing). User identity verified via Instagram user ID.

2. **Data:** Stores only ValueSkins-specific data (deals, contracts, reputation). Does not duplicate Instagram followers, posts, or engagement data. Queries Instagram Graph API on-demand for follower audit.
 3. **Payments:** Does not move money. Emits events to Meta payment infrastructure. Meta payment rail handles fund movement.
 4. **Moderation:** Reputation data reacts to Instagram account bans/strikes (scheduled job checks Instagram API, auto-flags creators with banned/restricted accounts).
 5. **Feature Flags:** Can be enabled/disabled per user, region, or surface (Reels tab, Creator Studio, Business Suite).
 6. **Rollback:** Feature disable is instant (no data migration needed). All ValueSkins data is independent and can be archived.
 7. **Backward Compatibility:** Old API versions remain queryable. Contracts and reputation exports signed and versioned. Old signatures remain valid forever.
-

CONCLUSION

This invention provides a comprehensive, deterministic, and scalable system to replace agency intermediaries in creator-brand collaboration. By combining server-authoritative state machines, cryptographic contracts, append-only audit trails, and portable reputation, ValueSkins enables transparent, trustworthy, and efficient marketplace interactions at any scale.

The system is production-ready for Meta-scale adoption and compatible with regulatory requirements across jurisdictions.